

题解

预期难度 LA<FB<CG<EH<MIJ<DK

实际过题 L<ABG<FE<CH<MJ<IDK

A 昨日重现

学校名最长为 4，可以写一个四重循环判断有多少个对应学校的子序列出现：

```
int ty2(string t) {  
    //t 是我们要找的长度为四的学校简称  
    int res = 0;  
    for(int i = 0; i < n; i++)  
        for(int j = i + 1; j < n; j++)  
            for(int k = j + 1; k < n; k++)  
                for(int l = k + 1; l < n; l++) {  
                    if(s[i] == t[0] && s[j] == t[1] && s[k] == t[2] && s[l] ==  
t[3])  
                        ++res;  
                }  
    return res;  
}
```

对于 WHU 写个三重循环一样求就可以了，时间复杂度 $O(n^4)$

参考代码：

```
#include<bits/stdc++.h>  
using namespace std;  
string s;  
int n;  
int ty1(string t) {  
    int res = 0;  
    for(int i = 0; i < n; i++)  
        for(int j = i + 1; j < n; j++)  
            for(int k = j + 1; k < n; k++) {  
                if(s[i] == t[0] && s[j] == t[1] && s[k] == t[2])  
                    ++res;  
            }  
    return res;  
}  
int ty2(string t) {  
    int res = 0;  
    for(int i = 0; i < n; i++)  
        for(int j = i + 1; j < n; j++)  
            for(int k = j + 1; k < n; k++)  
                for(int l = k + 1; l < n; l++) {  
                    if(s[i] == t[0] && s[j] == t[1] && s[k] == t[2] && s[l] ==  
t[3])  
                        ++res;  
                }  
    return res;  
}
```

```

    return res;
}
int main() {
    cin >> s;
    n = s.length();
    string s[5] = {"CCNU", "HUST", "HZAU", "WHU", "WHUT"};
    int ans = -1, id = 0;
    for(int i = 0; i < 5; i++) {
        int num;
        if(s[i].length() == 3) {
            num = ty1(s[i]);
        }
        else num = ty2(s[i]);
        if(num > ans)
            ans = num, id = i;
    }
    cout << s[id] << ' ' << ans << "\n";
    return 0;
}

```

轶事：这题因为出题人造数据的时候用的是 `WUT`，之后改过来之后没有重新造数据，导致没有答案为 `WHUT` 的数据，结果就是一些不忠诚的理兵邪恶的通过了本题，不好意思喵。

B 易守难攻

首先，如果 $n < 3$ 或者 $m < 3$ ，是无解的。这是因为我们无法找到不位于边界的格子。

当 $n \geq 3$ 且 $m \geq 3$ 时，答案的上界是 $\lfloor \frac{n-1}{2} \rfloor \times \lfloor \frac{m-1}{2} \rfloor$ 。证明如下：

- 令 $B_{r,c} = \{2r, 2r+1\} \times \{2c, 2c+1\}$ ，对于满足 $1 < x_i < n$ 且 $1 < y_i < m$ 的 (x_1, y_1) , (x_2, y_2) ，如果存在 r_0, c_0 ，使得 $(x_1, y_1) \in B_{r_0, c_0}$ 且 $(x_2, y_2) \in B_{r_0, c_0}$ ，那么 (x_1, y_1) 与 (x_2, y_2) 不可能同时被选择为驻地点。这是因为它们相邻，与各自大于相邻格子的高度值矛盾。
- 对于 $n \times m$ 的平原， $1 \leq r \leq \lfloor \frac{n-1}{2} \rfloor$, $1 \leq c \leq \lfloor \frac{m-1}{2} \rfloor$ 。这意味着驻地点的上界为 $\lfloor \frac{n-1}{2} \rfloor \times \lfloor \frac{m-1}{2} \rfloor$ 。

一种可行的构造方案是：将 $n \times m - k + 1, \dots, n \times m$ 依次放入 $(2i, 2j)$ 中，然后将 $1, 2, \dots, n \times m - k$ 从左到右，从上到下填满未填的格子即可。

C 故障机器人

题意：

一个字符串 S ， q 次独立询问，每次给出坐标 (x, y) 且忽略 l 到 r 这段进行移动，问是否能经过点 (x, y) 。

考察点：

前缀和，*STL*

题解：

可以视执行一次移动需要耗费一个单位时间，考虑按照 S 执行时，把 **所有能经过的点** 的 **到达时间** 按顺序记录下来，对于每个询问，想判断是否经过这次询问的 (x, y) 点：

- 如果在时间 l 之前，直接判断经过 (x, y) 与否即可
- 如果在时间 r 之后，则需要判断是否经过 (x', y') ，其中 $x' = x + (l \text{ 到 } r \text{ 时间内在 } x \text{ 方向上的位移})$ ， $y' = y + (l \text{ 到 } r \text{ 时间内在 } y \text{ 方向上的位移})$ 。因为如果可以到达点 (x', y') ，在忽略了 l 到 r 这段时间后，相当于回退了这段时间内的位移，那么一定会到达 (x, y) 。

不妨反向思考一下，可以直接查询第一次到达点 (x, y) 的时间是否小于 l 以及最后一次到达点 (x', y') 的时间是否大于 r 。其中经过点及其时间的记录可以很直观地用 `std::map < pair < int, int >, vector < int > >` 实现， l 到 r 时间内两个方向上的位移用前缀和处理后可以 $O(1)$ 得到。

总时间复杂度为 $O((n + q)\log n)$ 。

轶事：出题人在出此题时既没有玩过《杀戮尖塔》也没有看过《mygo》系列

D 死鱼工厂

考虑将每个点拆成 a_i 个点进行一对一的匹配问题。相比于经典的一对一匹配问题，本题多了一个来自相同的 a_i 的点不能匹配相同的 b_j 的限制。一对一匹配问题常用 KM 算法来解决。对于本题，我们只需要在 KM 算法 BFS 找增广路的时候，不走会造成不合法方案的边即可。时间复杂度 $O((\sum b_i)^3)$ 。

当然，也可以对费用流进行对偶后，并在 dij 算法跑最短路的时候用插入 $O(1)$ 的数据结构，即可做到 $((n + m)^2 \sum b_i)$ 。

E Allen 的树形地铺

首先可以在线地回答某个询问。以任意节点为根，不难发现答案其实只有2种情况：

- v 不在以 u 为根的子树中：这种情况答案为 $n - \text{siz}_u$ ，其中 siz_u 为 u 的子树大小。
- v 在以 u 为根的子树中：这时需要找到 v 在 u 的哪个子节点的子树中。设 v 在 w 的子树中，且 u 为 w 的父节点，则答案为 siz_w 。

首先可以采用 dfs 序或求 LCA 的方式判断是哪一种情况。对于第1种情况，可以 $O(1)$ 回答；第2种情况可以使用倍增或 dfs 序查询的方式 $O(\log n)$ 回答。

时间复杂度 $O(n + q \log n)$ 。

也可以采用离线的方式，将每个询问挂在节点 u 上。dfs 的过程中利用 dfs 序即可回答每个询问。时间复杂度 $O(n + q \log n)$ 或 $O(n + q)$ 。实现起来也不难。

F Anon 的疑问

注意到询问的最大值最大是 2×10^7 ，我们考虑预处理所有的值是否可以被分解为平方和，我们可以枚举第一维 i 和第二维 j ，将所有的 $i^2 + j^2$ 设置为可以被分解，可以注意到 i 和 j 都最多枚举到 $\sqrt{2 \times 10^7}$ ，之后可以 $O(1)$ 回答，因此时间复杂度为 $O(q + \sqrt{2 \times 10^7} * \sqrt{2 \times 10^7})$ ，即 $O(q + \max n_i)$ ，可以通过本题。

轶事：本题原来数据范围为 $n = 1e6$ ，因为赛时机子太快了故紧急修改为 $n = 2e7$ 。

G 你好，世界

题意

本题要求构造一个指令序列，使得 N 位计数器在每一步的预测均出错。

计数器机制：

1. **阈值**： $T = 2^{N-1}$ 。
2. **预测**：计数器当前值 $C < T$ 时预测为 0，否则预测为 1。
3. **更新**：实际为 1 则 $C = \min(C + 1, 2^N - 1)$ ；实际为 0 则 $C = \max(C - 1, 0)$ 。

构造策略

为了实现全过程预测错误（即 $E_k = k$ ），我们需要实时给出与预测结果相反的输入：

1. 前缀部分
 - 初始状态： $C = 0$ ，预测为 0。
 - 对抗操作：输入 1。
 - 过程：连续输入 2^{N-1} 个 1。在此期间，计数器从 0 逐步增加至 2^{N-1} ，但只要 $C < 2^{N-1}$ ，预测始终为 0，故全部预测错误。
2. 循环部分
 - 状态切换：当 $C = 2^{N-1}$ 时，预测切换为 1。
 - 对抗操作：输入 0。计数器减至 $2^{N-1} - 1$ ，预测切回 0。
 - 对抗操作：输入 1。计数器增至 2^{N-1} ，预测切回 1。
 - 规律：后续序列以 01 交替循环，使计数器在 2^{N-1} 与 $2^{N-1} - 1$ 之间反复横跳，保证每次预测均与实际相反。

结论

最终构造出的序列 S 结构为：

- **前 2^{N-1} 位**：全 1。
- **后续位**：从 0 开始的 01 循环。

复杂度分析：时间复杂度 $O(len)$ ，空间复杂度 $O(len)$ 。

H 哆啦 A 梦的神奇铜锣烧储存系统

哆啦 A 梦选出 m 个宝箱之后，剩下 $n - m$ 个宝箱，含有铜锣烧的期望为： $k \times \frac{n-m}{n}$ 。

系统从这 $n - m$ 个宝箱中，随机打开**空宝箱**，意味剩余 q 个宝箱里的铜锣烧数量，等于初始时 $n - m$ 里含有的铜锣烧数量。

答案即 $k \times \frac{n-m}{n}$ ，注意逆元。

I 尼克狐的旅行计划

求 LIS 的经典做法是维护一个单调栈，然后从左到右扫，添加一个新元素时，找到栈中第一个比它大的元素，并用这个元素替换；如果找不到，则加到栈顶。最后栈的元素个数就是 LIS 长度。

现在我们要 DP 这个东西。一个自然的想法是把这个栈的状态压成一个 n 位 01 串 s 。 dp 状态就是 $f[i][s]$ 表示前 i 个数中取了若干，栈中的情况是 s 的方案数。

然而 n 是 35，直接这么搞的话状态数比较大。需要优化。

观察一下转移的过程，做到第 i 个数时，将来会被插入的数字只有 $a[i+1], a[i+2], \dots, a[n]$ ，因此将来的转移只和栈中数字与这些数字的大小关系有关。把 $a[i+1..n]$ 的数字排序后设为 $b[1] < b[2] < \dots < b[n-i]$ ，那么 dp 状态实际上只需要记录栈中的数字在区间 $(0, b[1]), (b[1], b[2]), (b[2], b[3]), \dots$ 里分别有几个，而不需要具体记录是哪些数字，因为目前在同一个区间内的数字在将来的转移之中都是等价的。

这样优化之后状态数就会大大减少，足以通过。为了加速可以使用 $hash$ 表代替 map ，也可以在转移时使用位运算进行优化。

关于复杂度分析，可以证明最坏情况下，总的状态数是将 n 拆成若干个整数的最大乘积的级别，也就是 $O(3^{n/3})$ 。

J 彩票

对于一张的彩票，假设其最终与幸运数字的交集大小为 s ，那么对于任意的 $1 \leq i \leq s$ ，一定有恰好 $\binom{s}{i}$ 个大小为 i 的数集同时被彩票和幸运数字包含。

根据上述性质对 a_i 进行拆解。

设 b_i 表示：若存在一个大小为 i 的集合同时被彩票和幸运数字包含，那么就直接获得 b_i 枚金币。

那么令 $\sum_{j=1}^i \binom{i}{j} b_j = a_i$ ，即可将 "恰好有 i 个数字相同" 的限制拆解为 "至少有 j 个数字相同"。

b_i 数组根据上面的等式容易递推算出。

接下来只需要对于每个集合 S 统计出有多少张彩票包含了这个集合，设 f_S 表示包含集合 S 的彩票张数与 $b_{|S|}$ 的乘积，那么假设幸运数字的数集为 T ，则这张彩票对应的中奖总金额就是：
$$\sum_{S \subseteq T} f_S$$

每个 S 被多少张彩票包含和 f_S 之和分别可以用一次高维前缀和算出。

总复杂度为 $O(m2^m + nk)$ 。

K 多重集

结论是对于相同的左端点 l ，右端点越大， $[l, r]$ 对应的多重集就越小。

我们对于每个左端点 l 维护一个 $[L[l], R[l]]$ 表示对于左端点 l 来说可能的右端点范围，初始 $L[l] = l, R[l] = n$

然后我们每次随机的选取一个左端点端点 p ，再在 $[L[p], R[p]]$ 之间选取一个随机的右端点 x ，以 $[p, x]$ 作为参考段，求出有多少个子段比参考段大（对于每个左端点 i 找最远的满足 $[i, r[i]]$ 大于参考段的 $r[i]$ ， $r[i]$ 根据结论应该有单调性，可以快速求出）

如果 $\geq K$ 个大于参考段，那么就说明我们选择的参考段过小，我们让所有的 $R[i]$ 向 $r[i]$ 取 $\min$ 即可，同样的，如果我们选的参考段太大，我们就让所有的 $L[i]$ 向 $r[i] + 1$ 取 $\max$ 。这样随机选择大几十次，类似于快速排序的期望时间复杂度，随机选择排除解空间的速度也大概是 $O(\log n)$ 的。

L 那我问你

- 注意空格和大小写
- 注意不要拼写错误
- 复制题目中提供的文本作为参考是好习惯，但是不要因为复制了文本就省略掉检查
- 直接复制 *pdf* 是错误的选择，应该去 *DOMjudge* 提供的样例里复制

M 炫酷魔法

考虑二分最后的答案 x 。我们把问题转化为：给定一个实数 x ，判断是否存在满足

$$\frac{1}{k} \sum_{i=1}^k \gcd(a_{p_{i-1}+1}, a_{p_{i-1}+2}, \dots, a_{p_i}) \geq x$$

的划分方案。上式等价于：

$$\sum_{i=1}^k (\gcd(a_{p_{i-1}+1}, a_{p_{i-1}+2}, \dots, a_{p_i}) - x) \geq 0.$$

注意到合法性与 k 无关，所以我们不关心划分的段数。

设 dp_i 表示在前 i 个数中进行划分， $\sum_{i=1}^k (\gcd(a_{p_{i-1}+1}, a_{p_{i-1}+2}, \dots, a_{p_i}) - x)$ 的最大值。存在从 $1 \leq j < i$ 到 i 的转移：

$$dp_i \leftarrow \max_{j=1}^{i-1} \{dp_j + \gcd(a_{j+1}, \dots, a_i)\}$$

区间最大公因数可以使用 ST 表预处理，上述转移可以做到在 $O(n^2 \log V)$ 的时间内解决问题。

考虑优化转移方式。由于对于每个以 a_i 结尾的后缀 $[a_j, \dots, a_i]$ ，区间最大公因数的数目是对数级别的。对于区间最大公因数相同的后缀，我们只需要记录对应的最优 dp 值即可。通过预处理以 a_i 结尾的第 k 个不同的区间最大公因数的值可以做到 $O(n \log^2 V)$ 。